

Four-Device MongoDB Experiment

Client-Centric Consistency Guide and Analysis

Three Windows database nodes + one macOS workload client

Database tier	Three Windows laptops, one MongoDB 8.0 Docker container per laptop
Client tier	One macOS laptop running Python and PyMongo only
Replica set	rs0: 1 PRIMARY + 2 SECONDARY
Network	Hotspot LAN: Windows .2/.3/.4; macOS client .5
Models	Read-your-writes, monotonic reads, monotonic writes, writes-follow-reads

Version 2.0 - four-device architecture - 2026-09-06

Scope and contents

This guide continues from the existing MongoDB Three-Laptop Replica Set Setup, but moves the workload to a fourth macOS laptop. The three Windows laptops remain database replica members, each running one MongoDB 8.0 Docker container in replica set rs0. The Mac runs Python and PyMongo only and represents the sole logical client. This separation makes it possible to partition database members while keeping the client connected.

Important: run all fault-injection commands and test writes only in the course database. Do not use a production database or Docker volume containing important data.

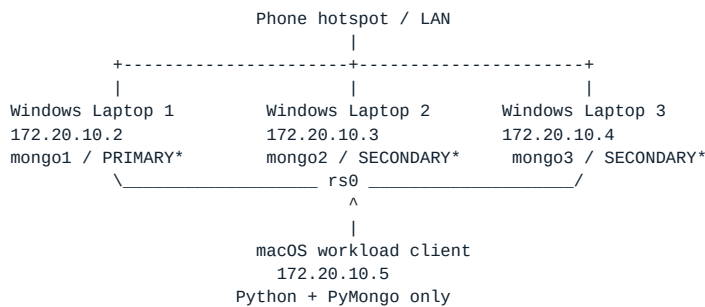
0. Four-device architecture and communication requirements

1. Research questions, definitions, and invariants
2. MongoDB parameters and predictions
3. One-time preparation: Python, member tags, and logging
4. Experiment A: Read-your-writes
5. Experiment B: Monotonic reads
6. Experiment C: Monotonic writes
7. Experiment D: Writes-follow-reads
8. Node-failure and network-partition scenarios
9. Data collection and report structure
10. Analysis and expected findings

Appendix A: Python experiment harness

Appendix B: References and AI-use statement

0. Four-device architecture and communication requirements



* Roles can change after an election.

The Mac is outside the replica set. It issues all workload operations, keeps client-session state, records JSONL evidence, and remains reachable when Windows members are isolated from one another. This is methodologically cleaner than running the client on a database host because client connectivity and replication connectivity can be controlled independently.

Device	Example IP	Role	Required software
Windows Laptop 1	172.20.10.2	MongoDB member n1	Docker Desktop; mongo1
Windows Laptop 2	172.20.10.3	MongoDB member n2	Docker Desktop; mongo2
Windows Laptop 3	172.20.10.4	MongoDB member n3	Docker Desktop; mongo3
macOS Laptop	172.20.10.5	Logical workload client	Python 3; PyMongo; experiment scripts

Required checkpoint before experimentation

- The four laptops are on the same subnet and the hotspot does not enable client isolation.
- The Mac can open TCP 27017 to 172.20.10.2, .3, and .4.
- The three Windows members can open TCP 27017 to one another.
- docker ps shows mongo1, mongo2, and mongo3 on their respective laptops.
- rs.status() reports one PRIMARY and two SECONDARY members.
- A document written to the PRIMARY is readable from both SECONDARY members.
- After the PRIMARY container stops, the other two members elect a new PRIMARY; the restarted node rejoins as SECONDARY.

1. Research questions, definitions, and invariants

Client-centric consistency concerns the sequence observed by one logical client. Here, the macOS Python workload is that client; the three Windows laptops are only database replica members. Strong tests keep causal context in one causally consistent ClientSession or explicitly transfer clusterTime and operationTime between client processes.

Model	Client history	Invariant	Counterexample
Read-your-writes (RYW)	W(v) -> R()	R.version >= v	The client writes v, then reads an older version
Monotonic reads (MR)	R1() -> R2()	R2.version >= R1.version	The second read is older than the first
Monotonic writes (MW)	W1 -> W2	W1 must remain before W2	W2 remains, but acknowledged W1 disappears
Writes-follow-reads (WFR)	R(source=v) -> W(derived=v)	The write follows the observed state	derived refers to v, but source rolls back below v

Three possible outcomes

Outcome	Interpretation	Consistency violation?
An old value or broken causal dependency is returned	A successful operation violates the invariant	Yes
Timeout, ServerSelectionError, or NotWritablePrimary	The system refuses or cannot finish the operation; availability falls	No
No old value is observed	No counterexample occurred in this trial	This does not prove a guarantee

Consistency models constrain successful operations. During a partition, a strong configuration may preserve consistency by waiting or returning an error. Record this as an availability result, not a consistency violation.

2. MongoDB parameters and predictions

Parameter	Values in this guide	Purpose
writeConcern	w:1; w:'majority' + j:true	Controls when a write is acknowledged; w:1 may roll back after failover
readConcern	local; majority	local may expose non-majority data; majority reads the majority-committed snapshot
readPreference	primary; secondary + tag	Routes reads to the PRIMARY or a selected SECONDARY
ClientSession	causal_consistency=True / False	The driver carries afterClusterTime and preserves causal order

MongoDB guarantee matrix for a causally consistent session

readConcern	writeConcern	RYW	MR	MW	WFR
majority	majority	Yes	Yes	Yes	Yes
majority	w:1	No	Yes	No	Yes
local	w:1	No	No	No	No
local	majority	No	No	Yes	No

No means that MongoDB does not guarantee the model. It does not mean every run must violate it. Replication is usually fast during normal operation, so weak configurations often appear to pass. A reproducible experiment must deliberately create a stale SECONDARY or a rollback.

3. One-time preparation on macOS

3.1 Install PyMongo on the Mac client

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install "pymongo>=4.10,<5"
python -c "import pymongo; print(pymongo.version)"
```

3.2 Verify Mac-to-member connectivity

```
ipconfig getifaddr en0
nc -vz 172.20.10.2 27017
nc -vz 172.20.10.3 27017
nc -vz 172.20.10.4 27017
```

All three nc tests must succeed. Ping may be blocked by Windows even when TCP works, so TCP 27017 is the decisive check. If a test fails, verify Docker port mapping, --bind_ip_all, the Windows inbound firewall rule, and hotspot client isolation.

3.3 Configure the replica-set URI

```
export MONGO_URI='mongodb://172.20.10.2:27017,172.20.10.3:27017,172.20.10.4:27017/?replicaSet=rs0'
python mongo_consistency_lab.py topology
```

```
# Optional direct-member check without a long environment variable
python mongo_consistency_lab.py topology \
  --uri 'mongodb://172.20.10.2:27017/?directConnection=true' \
  --uri 'mongodb://172.20.10.3:27017/?directConnection=true' \
  --uri 'mongodb://172.20.10.4:27017/?directConnection=true'
```

If the hotspot assigns new addresses, run ipconfig on the Windows laptops and ipconfig getifaddr en0 on the Mac, then update the deployment. PyMongo discovers members using the addresses published by rs.conf(), so every published member address must be reachable from the Mac.

The current course deployment has no database authentication. Use only a trusted private hotspot and never expose TCP 27017 to a public network. If authentication is later enabled, add credentials and authSource=admin to every URI.

3.4 Add member tags for deterministic reads

In mongosh, inspect rs.conf() and match each members array index to its host before making the following change.

```
cfg = rs.conf()
cfg.members[0].tags = { labNode: "n1" }
cfg.members[1].tags = { labNode: "n2" }
cfg.members[2].tags = { labNode: "n3" }
rs.reconfig(cfg)
rs.conf()
```

Do not assume members[0] is always the current PRIMARY. priority=2 makes Laptop 1 more likely to become PRIMARY, but roles can change after a fault.

3.5 Record the same evidence in every trial

Field	Example or meaning
run_id	ryw-weak-partition-01; unique for every trial
Configuration	readConcern, writeConcern, readPreference, causal session
Operation	W1, R1, R2, W2; start/end UTC and elapsed_ms
Member	hello().me, primary, isWritablePrimary, secondary
Result	version, acknowledged, error type, violation
Fault timeline	Exact partition/stop/recovery times and matching rs.status() output

4. Experiment A: Read-your-writes consistency

A0 - Normal-operation baseline

```
python mongo_consistency_lab.py prepare --run ryw-normal-01
python mongo_consistency_lab.py ryw-strong --run ryw-normal-01 --tag n2
```

Expected: both operations succeed and `read_version >= write_version`. Use the latency as a baseline for the partition test.

A1 - Weak configuration: read from a stale SECONDARY

- Run prepare and verify that all three members store `version=0`.
- Choose n2 as the stale SECONDARY. Block n2's replication and heartbeat traffic to n1/n3, but retain workload-client access to n2:27017.
- The PRIMARY and the other SECONDARY still form a majority, so the majority write can succeed.
- Write a nanosecond timestamp version, then use `directConnection` with `readConcern=local` to read from n2.

```
python mongo_consistency_lab.py prepare --run ryw-weak-partition-01
# After isolating n2
python mongo_consistency_lab.py ryw-weak --run ryw-weak-partition-01 \
  --stale-uri "mongodb://172.20.10.3:27017/?directConnection=true"
```

Expected counterexample: `write_version` is the new timestamp but `read_version` remains 0, so `violation=true`. A majority write alone does not provide RYW when the later read is local, non-causal, and routed to a stale SECONDARY.

A2 - Strong configuration: causally read the same stale member

```
python mongo_consistency_lab.py prepare --run ryw-strong-partition-01
# After isolating n2; the tag must identify n2
python mongo_consistency_lab.py ryw-strong --run ryw-strong-partition-01 --tag n2
```

Expected: the read does not return the old value. It times out if the partition lasts beyond 10 seconds. If the network is restored while the read waits, n2 returns only after reaching the session's `afterClusterTime`. Classify timeout as loss of availability, not a violation.

5. Experiment B: Monotonic-reads consistency

The same logical client reads twice: first from the PRIMARY after seeing the new version, then from a stale SECONDARY. A lower second version is an observable reversal of time.

B1 - Weak configuration

```
python mongo_consistency_lab.py prepare --run mr-weak-partition-01
# After isolating n2
python mongo_consistency_lab.py mr-weak --run mr-weak-partition-01 \
  --stale-uri "mongodb://172.20.10.3:27017/?directConnection=true"
```

The script writes a new version with majority, reads R1 from the PRIMARY, and then reads R2 from n2 with `readConcern=local`. Expected: `R2 < R1` and `violation=true`.

B2 - Strong configuration

```
python mongo_consistency_lab.py prepare --run mr-strong-partition-01
# After isolating n2
python mongo_consistency_lab.py mr-strong --run mr-strong-partition-01 --tag n2
```

The reads occur in one causal session: first from PRIMARY, then from n2. The second read must wait until n2 reaches the session's causal time. It therefore returns the same/newer version or times out.

Do not rely on `secondaryPreferred` without a tag. It may randomly select the newer SECONDARY and hide the counterexample. A member tag makes n2 the only eligible target.

6. Experiment C: Monotonic-writes consistency

MongoDB sends ordinary writes to the PRIMARY and replicates them in oplog order, so MW is difficult to violate during normal operation. The weak counterexample uses a w:1 write that is acknowledged, followed by failover before replication and then rollback.

Stage	Action	Required evidence
1	Isolate the old PRIMARY from both SECONDARY members while the workload can still reach it	The old PRIMARY briefly continues accepting w:1
2	Write W1 directly to the old PRIMARY with w:1, j=false	W1 has ok=true in results.jsonl
3	Immediately stop the old PRIMARY and restore communication between the other two members	The other two elect a new PRIMARY
4	Write W2 through the replica-set URI with majority	W2 has ok=true
5	Restore the old member and network; wait for rollback and catch-up	Final state contains W2 but not W1

```
python mongo_consistency_lab.py prepare --run mw-weak-failover-01
```

```
# Immediately after isolation; replace the IP with the actual old PRIMARY
```

```
python mongo_consistency_lab.py mw-write --run mw-weak-failover-01 \
  --step 1 --wc one \
  --uri "mongodb://172.20.10.2:27017/?directConnection=true"
```

```
# On the old PRIMARY laptop: docker stop mongo1
```

```
# After the other two members elect a PRIMARY
```

```
python mongo_consistency_lab.py mw-write --run mw-weak-failover-01 --step 2 --wc majority
```

```
# docker start mongo1; restore networking; wait for SECONDARY
```

```
python mongo_consistency_lab.py mw-check --run mw-weak-failover-01
```

A valid counterexample requires W1 ok=true, W2 ok=true, w1_present=false, and w2_present=true. If W1 returns NotWritablePrimary, the old node had already stepped down; discard that trial and restart from prepare.

C2 - Strong control

```
python mongo_consistency_lab.py prepare --run mw-strong-failover-01
```

```
python mongo_consistency_lab.py mw-write --run mw-strong-failover-01 --step 1 --wc majority
```

```
# After W1 succeeds, stop the current PRIMARY and wait for election
```

```
python mongo_consistency_lab.py mw-write --run mw-strong-failover-01 --step 2 --wc majority
```

```
# Restore the original node, wait for catch-up, then check
```

```
python mongo_consistency_lab.py mw-check --run mw-strong-failover-01
```

Expected: W1 and W2 both remain. A majority write returns only after a majority of voting members acknowledge it, so an ordinary single-node failure does not roll back successful W1.

7. Experiment D: Writes-follow-reads consistency

Use a source document and a derived document. The client reads source.version=v and writes derived.source_version=v. If v is rolled back while derived remains on the new PRIMARY, the write has survived without the state on which it depended.

D1 - Weak configuration

```
python mongo_consistency_lab.py prepare --run wfr-weak-partition-01
```

```
# Immediately after isolating the old PRIMARY, write and read source
```

```
python mongo_consistency_lab.py wfr-read --run wfr-weak-partition-01 \
  --wc one --rc local --token state/wfr-weak.json \
  --uri "mongodb://172.20.10.2:27017/?directConnection=true"
```

```
# Stop old PRIMARY; after election, write derived on the new PRIMARY
```

```
python mongo_consistency_lab.py wfr-write --wc majority --token state/wfr-weak.json
```

```
# Restore the old member and network; wait for catch-up
```

```
python mongo_consistency_lab.py wfr-check --run wfr-weak-partition-01
```

Expected: `derived_source_version` is greater than `source_version_now`, and `violation=true`. As in MW, phase 1 must have been acknowledged for the trial to count.

D2 - Strong configuration with transferred causal metadata

```
python mongo_consistency_lab.py prepare --run wfr-strong-failover-01
python mongo_consistency_lab.py wfr-read --run wfr-strong-failover-01 \
  --wc majority --rc majority --causal --token state/wfr-strong.json

# After phase 1 succeeds, stop PRIMARY and wait for election
python mongo_consistency_lab.py wfr-write --wc majority --resume-causal \
  --token state/wfr-strong.json

# Restore the member and check
python mongo_consistency_lab.py wfr-check --run wfr-strong-failover-01
```

The token stores `clusterTime` and `operationTime` in BSON Extended JSON. The new process advances its session to those values before writing. Expected: source does not roll back and the derived dependency remains valid.

8. Node-failure and network-partition scenarios

8.1 Single-node failure with Docker

Run Docker commands on the Windows laptop hosting the current PRIMARY, while the Python workload continues on the Mac. Always verify the current role with `rs.status()` or the Mac topology command; do not infer it only from the container name.

```
docker stop mongo1
# From another Windows member, wait for rs.status() to show a new PRIMARY
docker start mongo1
# Wait until mongo1 rejoins as SECONDARY
```

With two voting members still available, the replica set retains a majority and can elect a PRIMARY. The Mac stays outside the failure domain and records the interval from `docker stop` until the new PRIMARY accepts writes, plus `AutoReconnect`, `ServerSelectionTimeout`, and successful request counts.

8.2 Recommended network-partition method

The current Windows Firewall rule allows `LocalSubnet` access to TCP 27017. Do not delete it. Add temporary and narrowly scoped Block rules, then remove them after each trial. The `RemoteAddress` list must distinguish database peers from the workload client.

```
# Administrator PowerShell on Laptop 2: block MongoDB peer traffic from Laptops 1 and 3
New-NetFirewallRule -DisplayName "DSA5208_Block_RS_In" -Direction Inbound `
  -Protocol TCP -LocalPort 27017 -RemoteAddress 172.20.10.2,172.20.10.4 -Action Block

New-NetFirewallRule -DisplayName "DSA5208_Block_RS_Out" -Direction Outbound `
  -Protocol TCP -RemotePort 27017 -RemoteAddress 172.20.10.2,172.20.10.4 -Action Block

# Restore networking
Remove-NetFirewallRule -DisplayName "DSA5208_Block_RS_In"
Remove-NetFirewallRule -DisplayName "DSA5208_Block_RS_Out"
```

Because the Mac uses 172.20.10.5, these rules block only Windows database peers .2 and .4. The Mac should still reach n2 at .3 and read its stale state. Verify this from the Mac with `nc -vz 172.20.10.3 27017` and the `direct topology` command. If the hotspot changes addresses, rewrite the firewall rules before running the trial.

8.3 Fault profiles for the four-device design

Fault profile	Windows firewall action	Mac connectivity	Purpose
Stale n2 SECONDARY	On n2, block n1 (.2) and n3 (.4)	Mac .5 can still reach n2 .3	RYW and MR stale-read counterexamples
Old PRIMARY minority	On old PRIMARY, block both other members; add reciprocal blocks if needed	Mac .5 can still reach old PRIMARY	Create a brief w:1 write/read window before step-down
Single-node crash	No firewall change; docker stop current PRIMARY	Mac stays connected to remaining members	Measure election and rollback behavior

8.4 Minimum scenario matrix

Scenario	Weak configuration	Strong configuration	Main measurements
Normal	local / w:1 / no session	majority / majority / causal	Correctness and p50/p95 latency
Single-node failure	Lose PRIMARY immediately after w:1	Lose PRIMARY after majority acknowledgement	Rollback, election time, errors
Network partition	Target stale reads; old-PRIMARY w:1 window	majority + causal session	Violations, timeouts, recovery time

Run at least 20 trials per cell; 30 is preferable. Change one variable at a time and keep all other concerns, preferences, session settings, and fault procedures fixed.

9. Data collection and report structure

9.1 Raw trial table

run_id	Model	Scenario	Configuration	Success	Violation	Latency	Error	Note
ryw-w-01	RYW	n2 partition	local / maj / no session	Yes	Yes	4.8	-	R=0<W
ryw-s-01	RYW	n2 partition	maj / maj / causal	No	No	10012	ExecTimeout	availability

9.2 Summary table

Model/config	Scenario	Trials	Success	Violations	Timeouts/errors	p50	p95
RYW weak	partition	30	30	28	0	fill	fill
RYW strong	partition	30	4	0	26	fill	fill

The numbers above are layout examples and must be replaced with your observed data. Many strong-configuration timeouts are useful evidence of the consistency/availability trade-off, not failed experiments.

9.3 How to write the analysis

- Prediction: use the MongoDB guarantee matrix to state which models are and are not guaranteed.
- Mechanism: explain asynchronous replication, majority commit point, afterClusterTime, election, and rollback.
- Observation: report successful operations, violations, timeouts, errors, and latency across N trials.
- Comparison: say whether results match predictions. If no counterexample appears, write 'not observed', not 'proved'.
- Limitations: hotspot variability, clock synchronization, firewall granularity, the short old-PRIMARY write window, sample size, and driver retry behavior.
- Reproduction: include all versions, rs.conf(), connection method, scripts, and exact fault commands.

9.4 Suggested three-person division

Member	Primary responsibility	Cross-check
A	Cluster status, Docker faults, firewall timeline	Verify actual PRIMARY/SECONDARY in every trial
B	Python workload, session/concern settings, JSONL	Confirm which operations were acknowledged
C	Repeated trials, statistics, MongoDB logs, report	Independently recompute violation predicates

10. Analysis and expected findings

This chapter provides the analysis that should accompany the experiments. It distinguishes theoretical predictions from measured findings. Bracketed fields such as [N], [V], and [p95] must be replaced with values computed from your JSONL output; no measured result is assumed in this guide.

10.1 Metrics and denominators

```
RYW violation = 1 if a successful read returns R.version < W.version
MR violation  = 1 if successful reads return R2.version < R1.version
MW violation  = 1 if acknowledged W2 remains but acknowledged W1 disappears
WFR violation = 1 if derived exists and source.version < derived.source_version

violation_rate = violating successful histories / completed successful histories
availability    = completed histories / attempted histories
error_rate     = histories ending in timeout or driver/server error / attempted histories
latency_ms     = operation_end_monotonic - operation_start_monotonic
```

A timeout is excluded from the consistency-violation denominator because it did not return an inconsistent value, but it remains in the availability denominator. For MW and WFR, count a rollback counterexample only when all prerequisite writes were acknowledged. Report p50 and p95 latency separately for successful operations and timeouts.

10.2 Model-by-model predicted analysis

Model	Weak configuration analysis	Strong configuration analysis
RYW	The Mac writes a new version to the majority side, then reads n2 directly. Because n2 is isolated and rc=local, it can return version 0. This is a direct RYW violation.	The causal session attaches afterClusterTime. A tagged read from stale n2 waits for catch-up or times out; it must not return version 0.
MR	The Mac first reads the new version from PRIMARY, then reads isolated n2. If R2<R1, the same client observes time going backwards.	The second causal read cannot precede the first read's operation time. It returns an equal/newer version or loses availability.
MW	W1 acknowledged with w:1 on the old PRIMARY can be absent from the elected majority and later roll back, while W2 survives on the new PRIMARY.	W1 acknowledged with majority is present on an electable majority before failover. A later W2 cannot survive while W1 disappears under an ordinary single-node fault.
WFR	The Mac reads a non-majority source version from the old PRIMARY, then writes derived on the new PRIMARY. Rollback can remove the source state but leave the dependent write.	A majority read/write causal chain carries operation time into the dependent write. The dependency remains valid, or the write waits/fails.

10.3 Report-ready interpretation: RYW and MR

For the weak RYW configuration, the independent Mac client writes through the rs0 URI while n2 is disconnected from replication, then reads n2 through a direct connection. A returned value lower than the acknowledged write version demonstrates that write durability on a majority does not by itself make an unrelated local secondary read causally ordered. The violation is caused by asynchronous replica lag combined with non-causal routing, not by concurrent application writes.

The MR experiment changes only the client history: the Mac first observes the current value at the PRIMARY and then migrates its read to stale n2. A decrease from R1 to R2 is stronger evidence than merely reading an old value because it shows a single client's visible state regressing. Under majority read concern and a causal session, afterClusterTime converts that regression into waiting or an error. Therefore, an increase in timeout rate alongside zero stale returns supports the expected consistency-versus-availability trade-off.

10.4 Report-ready interpretation: MW and WFR

The weak MW history depends on the distinction between acknowledgement and majority commitment. A w:1 response proves only that the old PRIMARY accepted W1. If that member is isolated before replication, the two-member majority can elect a new PRIMARY whose history lacks W1. W2 can then be majority committed. When the old member rejoins, rollback removes W1, producing a final state in which W2 remains without W1. This is a valid counterexample only if the Mac log shows that both writes were acknowledged.

WFR uses the same rollback mechanism but exposes a different causal relationship. The Mac reads source version v from the isolated old PRIMARY and later writes a derived record that cites v on the new PRIMARY. If v rolls back while

the derived record survives, the final database contains an effect without the state observed by its cause. In the strong control, majority read/write concerns and transferred session times prevent this dependency gap; loss of quorum instead produces blocking or failure.

10.5 Cross-scenario analysis

Scenario	Expected consistency result	Expected availability/latency result	Interpretation
Normal operation	Weak and strong trials may both show zero observed violations because replicas catch up quickly	Strong concerns usually add acknowledgement/read latency	Absence of a weak counterexample is not proof of a guarantee
One-node failure	Majority-acknowledged histories remain consistent; an unreplicated w:1 write may be lost	Brief election errors and a latency spike are expected	Separates acknowledged-local writes from majority-committed writes
Stale-secondary partition	Weak RYW/MR can return old values; causal majority reads should not	Strong reads may wait until timeout	Shows consistency gained by sacrificing availability
Old-PRIMARY minority partition	Weak MW/WFR rollback counterexamples become possible	The old PRIMARY has only a short pre-step-down write window	Invalid trials must be discarded if W1 was not acknowledged

10.6 Text template for measured results

Replace every bracketed value after running the experiments:

- RYW: In [N] completed weak histories under the n2 partition, [V] returned a version lower than the Mac client's preceding write, for a violation rate of [V/N]. In the strong configuration, [S] reads completed consistently and [T] timed out; no completed read returned an older value. The result [agrees/does not agree] with the prediction.
- MR: Of [N] weak primary-to-n2 read migrations, [V] produced $R2 < R1$. The strong causal-session trials produced [V2] completed-history violations and [T] timeouts. The evidence indicates that causal ordering prevented state regression at the cost of [higher latency/lower availability].
- MW: [A] trials produced acknowledged W1 and W2 operations. In [V] valid weak trials, W1 disappeared after rollback while W2 remained. All [S] valid majority-control trials retained both writes. Trials where W1 returned NotWritablePrimary were classified as invalid rather than violations.
- WFR: Among [A] valid weak trials, [V] ended with source.version lower than derived.source_version. In [S] strong trials, the dependency remained valid whenever the dependent write completed; [T] operations timed out during loss of the required causal/majority state.

10.7 Limitations affecting interpretation

- Phone-hotspot scheduling and signal quality add uncontrolled latency and may change which member is considered nearest.
- The old PRIMARY normally steps down quickly after losing contact with a majority, so the w:1 counterexample has a narrow timing window and may require repeated trials.
- Windows Firewall rules operate at the host network boundary; Docker Desktop networking should be verified rather than assumed.
- The Mac's wall clock and the Windows clocks may differ. Use monotonic time for durations and program order/session times for causal order.
- A small number of trials can miss rare violations. Report confidence cautiously and preserve all invalid/error trials rather than selectively omitting them.
- The experiments study one replica set, one client, and controlled key/value histories. Results should not be generalized to sharded clusters, transactions, or concurrent multi-client workloads without additional tests.

Appendix A: Python experiment harness

The companion `mongo_consistency_lab.py` provides the commands below. Submit it with `requirements.txt`, a password-free `.env.example`, the result JSONL files, and reproduction instructions.

Command	Purpose
<code>topology</code>	Read <code>hello()</code> and verify the member reached by each URI
<code>prepare --run ID</code>	Delete only that run's test records and initialize <code>version=0</code> with majority
<code>ryw-weak / ryw-strong</code>	Run the weak/strong RYW comparison
<code>mr-weak / mr-strong</code>	Run the weak/strong MR comparison
<code>mw-write / mw-check</code>	Write <code>W1/W2</code> in phases and inspect post-rollback state
<code>wfr-read / wfr-write / wfr-check</code>	Persist/resume causal metadata and verify the dependency

Implementation choices

- `retryWrites=False` prevents an automatic retry from hiding the original failover behavior.
- `serverSelectionTimeoutMS` and `maxTimeMS` prevent indefinite waits during a partition.
- A nanosecond timestamp is used as version so regressions are easy to detect.
- `directConnection=true` is used only for a selected stale SECONDARY or old PRIMARY.
- The replica-set URI is used for normal writes and post-failover writes so the driver can discover the new PRIMARY.
- JSONL records UTC, elapsed ms, configuration, target member, version, error, and violation.

Do not use one `ClientSession` concurrently across threads. The basic tests are sequential histories of one logical client. For a concurrent extension, create a separate session and `client_id` per worker.

Suggested submission structure

```
submission/
  report.pdf
  README.md
  requirements.txt
  .env.example
  mongo_consistency_lab.py
  results/
    raw_results.jsonl
    summary.csv
  evidence/
    versions.txt
    rs_conf.json
    fault_timeline.md
```

Appendix B: References and AI-use statement

- [1] MongoDB Manual, Causal Consistency and Read and Write Concerns.
<https://www.mongodb.com/docs/manual/core/causal-consistency-read-write-concerns/>
- [2] MongoDB Manual, Read Concern. <https://www.mongodb.com/docs/manual/reference/read-concern/>
- [3] MongoDB Manual, Read Preference. <https://www.mongodb.com/docs/manual/core/read-preference/>
- [4] MongoDB Manual, Rollbacks During Replica Set Failover. <https://www.mongodb.com/docs/manual/core/replica-set-rollbacks/>
- [5] MongoDB PyMongo Driver, Transactions and Causal Consistency.
<https://www.mongodb.com/docs/languages/python/pymongo-driver/current/crud/transactions/>
- [6] MongoDB PyMongo Driver, Configure CRUD Operations.
<https://www.mongodb.com/docs/languages/python/pymongo-driver/current/crud/configure/>
- [7] Existing group setup document, MongoDB Three-Laptop Replica Set Setup, 2026-08-31.

Example AI-use statement

This project used OpenAI Codex to help review MongoDB documentation, design the initial experiment matrix, generate a Python experiment scaffold, and format the experiment guide. The group members manually verified the replica-set configuration, read/write parameters, fault-injection steps, script output, and final analysis. All experimental results reported in the submission were produced by the group. The final report should also identify the model/date and describe the exact scope of use required by the course.

Before finishing: remove every temporary Block firewall rule, restart all containers, confirm `rs.status()` returns to 1 PRIMARY + 2 SECONDARY, and back up `results.jsonl` and MongoDB logs.